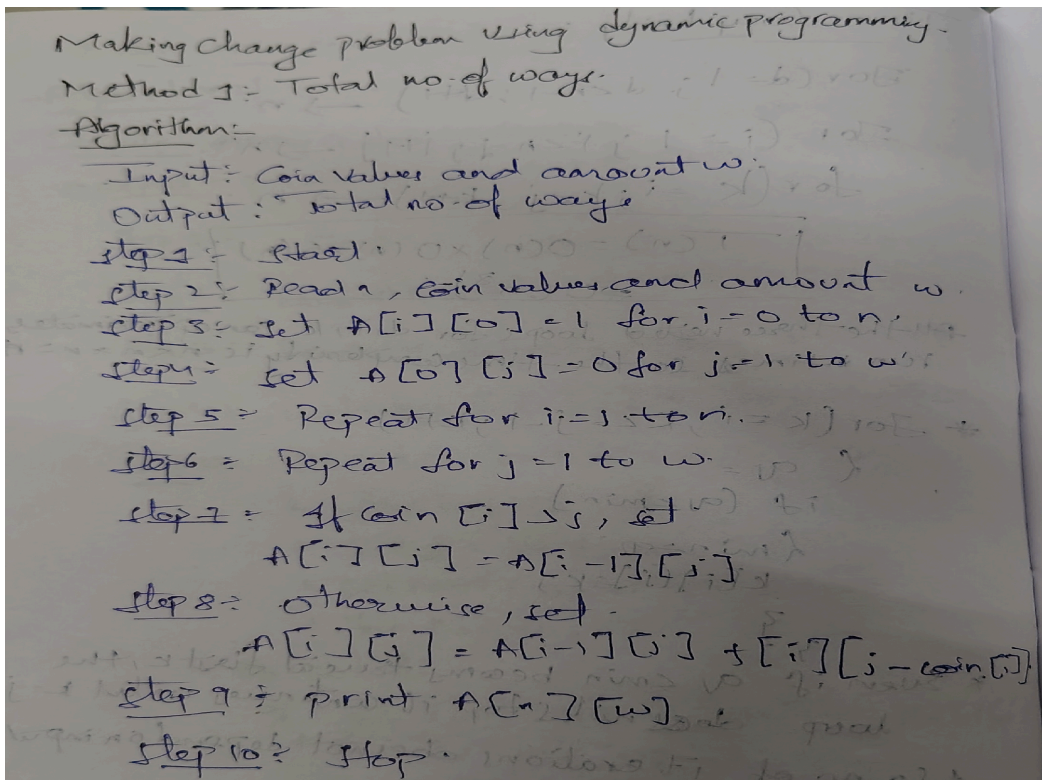


## PRACTICAL-7

**AIM:** Implementation of making a change problem using dynamic programming.

**METHOD1:** Total number of ways.

**ALGORITHM:**



Making change problem using dynamic programming.  
Method 1: Total no. of ways.  
Algorithm:-  
Input: Coin values and amount  $w$ .  
Output: Total no. of ways.  
Step 1: Start.  
Step 2: Read  $n$ , coin values and amount  $w$ .  
Step 3: Set  $A[i][0] = 1$  for  $i = 0$  to  $n$ .  
Step 4: Set  $A[0][j] = 0$  for  $j = 1$  to  $w$ .  
Step 5: Repeat for  $i = 1$  to  $n$ .  
Step 6: Repeat for  $j = 1$  to  $w$ .  
Step 7: If  $\text{coin}[i] \leq j$ ,  
 $A[i][j] = A[i-1][j] + A[i][j - \text{coin}[i]]$   
Step 8: Otherwise, set  
 $A[i][j] = A[i-1][j]$   
Step 9: Print  $A[n][w]$ .  
Step 10: Stop.

### CODE:

```
#include <iostream>
using namespace std;
class ChangeMaking {
private:
    int coins[10];
    int n;
    int w;
```

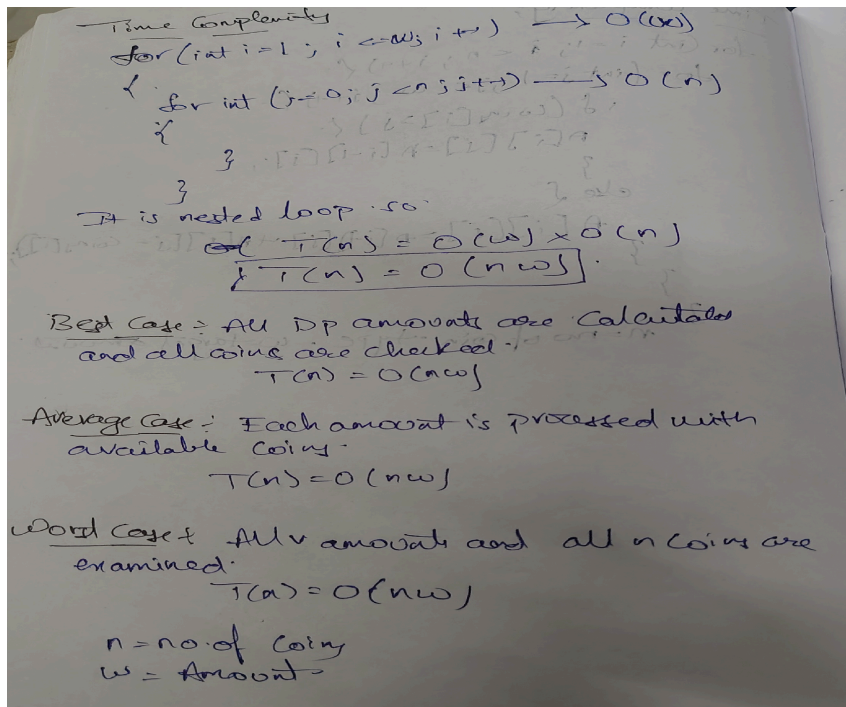
```
int A[10][100];
public:
void getInput() {
    cout << "Enter number of coin types : ";
    cin >> n;
    cout << "Enter coin values : ";
    for (int i = 1; i <= n; i++)
        cin >> coins[i];
    cout << "Enter amount : ";
    cin >> w;
}
void solve() {
    for (int i = 0; i <= n; i++)
        A[i][0] = 1;
    for (int j = 1; j <= w; j++)
        A[0][j] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= w; j++) {
            if (coins[i] > j)
                A[i][j] = A[i - 1][j];
            else
                A[i][j] = A[i - 1][j]
                    + A[i][j - coins[i]];
        }
    }
}
void printTable() {
    cout << "\nDP Table:\n";
    cout << "Coin ";
    for (int j = 0; j <= w; j++)
        cout << j << " ";
    cout << endl;
    for (int i = 0; i <= n; i++) {
        if (i == 0)
            cout << "0 ";
    else
        cout << coins[i] << " ";
        for (int j = 0; j <= w; j++)
            cout << A[i][j] << " ";
        cout << endl;
    }
}
```

```
}  
void showResult() {  
    cout << "\nTotal Number of Ways = "  
        << A[n][w] << endl;  
}  
};  
int main() {  
    ChangeMaking c;  
    c.getInput();  
    c.solve();  
    c.printTable();  
    c.showResult();  
    return 0;  
}
```

## OUTPUT:

```
Enter number of coin types : 5  
Enter coin values : 2  
6  
9  
8  
5  
Enter amount : 10  
  
DP Table:  
Coin 0  1  2  3  4  5  6  7  8  9  10  
0      1  0  0  0  0  0  0  0  0  0  0  
2      1  0  1  0  1  0  1  0  1  0  1  
6      1  0  1  0  1  0  2  0  2  0  2  
9      1  0  1  0  1  0  2  0  2  1  2  
8      1  0  1  0  1  0  2  0  3  1  3  
5      1  0  1  0  1  1  2  1  3  2  4  
  
Total Number of Ways = 4  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

## TIME COMPLEXITY:



## Method 2: Minimum number of ways

### ALGORITHM:

2) Minimum no. of ways  
Input: n coin denomination, target amount w  
Output: minimum no. of coins  
Step 1: Start  
Step 2: Read the number of coin denomination n.  
Step 3: Read the n coin values; store in array coins[]  
Step 4: Read the target amount w  
Step 5: Create a DP array dp[0..w]  
Step 6: dp[0] = 0  
Step 7: Initialize all other dp[i] values to (∞)  
Step 8: For i = 1 to w, do  
    \* For each coin [j], do  
    \* If coins[j] ≤ i, calculate:  
        dp[i] = min(dp[i], dp[i - coins[j]] + 1)  
Step 9: After completing the table check dp[w]  
Step 10: If dp[w] is still infinity display  
    "Change Cannot be made"  
Step 11: Otherwise, display dp[w] as the min  
    no. of coins required.  
Step 12: STOP

### CODE:

```
#include <iostream>
#include <algorithm>
using namespace std;

class ChangeMaking {
private:
    int coins[10];
    int n, w;
    int A[100][100];
```

```
public:
void getInput() {
    cout << "Enter number of coin types: ";
    cin >> n;

    cout << "Enter coin values: ";
    for (int i = 1; i <= n; i++) {
        cin >> coins[i];
    }

    cout << "Enter amount: ";
    cin >> w;
}

void solve() {
    for (int i = 0; i <= n; i++)
        A[i][0] = 0;

    for (int j = 1; j <= w; j++)
        A[0][j] = 999;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= w; j++) {
            if (coins[i] > j)
                A[i][j] = A[i - 1][j];
            else
                A[i][j] = min(A[i - 1][j],
                             1 + A[i][j - coins[i]]);
        }
    }
}

void printTable() {
    cout << "\nDP Table:\n";
    cout << "    ";

    for (int j = 0; j <= w; j++)
        cout << j << "    ";

    cout << endl;
```

```
for (int i = 0; i <= n; i++) {
    cout << "i=" << i << " ";

    for (int j = 0; j <= w; j++) {
        if (A[i][j] == 999)
            cout << "0 ";
        else
            cout << A[i][j] << " ";
    }

    cout << endl;
}

void showResult() {
    if (A[n][w] == 999)
        cout << "\nAmount cannot be formed.";
    else
        cout << "\nMinimum number of coins = "
            << A[n][w];
}

};

int main() {
    ChangeMaking c;
    c.getInput();
    c.solve();
    c.printTable();
    c.showResult();
    return 0;
}
```

## OUTPUT:

```
Enter number of coin types: 4
Enter coin values: 3
4
7
2
Enter amount: 7

DP Table:
  0  1  2  3  4  5  6  7
i=0 0  0  0  0  0  0  0  0
i=1 0  0  0  1  0  0  2  0
i=2 0  0  0  1  1  0  2  2
i=3 0  0  0  1  1  0  2  1
i=4 0  0  1  1  1  2  2  1

Minimum number of coins = 1
```

## TIME COMPLEXITY:

Time Complexity

for (int i=1; i<=W; i++) →  $W$   
 { for (int j=0; j<=n; j++) →  $n$   
 { if (coins[i] <= j) {  
 { dp[i] = min(dp[i], dp[j-coins[i]]+1);  
 }  
 }  
 }  
 $T(n) = O(n) \times O(W)$   
 $T(n) = O(nW)$

Best Case: The minimum amount can be found easily, the algorithm still finds DP table from 1 to W and check available coin  
 Best Case =  $O(nW)$

Average Case: The algorithm still processes DP states for all amounts and checks the coins  
 $T(n) = O(nW)$

Worst Case: Every amount from 1 to W is processed and every one of the coins is checked  
 $T(n) = O(nW)$

Department of Computer Engineering  
01AI0506 Design and Analysis of Algorithm Lab Manual

**CONCLUSION:** The Making Change Problem was successfully implemented using Dynamic Programming. The method stores solutions to smaller amounts and uses them to calculate the solution for the required amount, avoiding repeated calculations. For the minimum number of coins version, the time complexity is  $O(nw)$